

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 14. Functions

Lecture: 163. Iteration & Sequence Functions - Built-in

Section 1: Lecture Summary

This lecture introduces several important built-in Python functions related to **iteration and sequence manipulation**, focusing on common sequence types such as lists and tuples. It covers the **sorted()**, **reversed()**, **slice()**, and **iter()** functions. The instructor explains how these functions operate on sequences, their parameters, and how to use them effectively with examples using lists. Key points include how `sorted()` can sort sequences with optional parameters for order and sorting criteria, how `reversed()` returns a reverse iterator rather than a list, the use of the `slice` function to create reusable slice objects, and how to create and use iterators with `iter()` and `next()`. The lecture closes by emphasizing practice with these built-in functions to get comfortable using them.

Section 2: Key Concepts and Explanations

Sequence Types and Iteration

- Sequences are ordered collections such as lists, tuples, and sets.
- Iteration means going through elements of a sequence one by one.

sorted(iterable, /, , key=None, reverse=False)

- Takes any iterable and returns a **new sorted list**.
- `'key'`: a function applied to each element for sorting comparison (e.g., `'abs'` for absolute values).
- `'reverse'`: if `'True'`, returns the list in descending order; default is `'False'`.
- Does not modify the original iterable.

reversed(seq, /)

- Takes a sequence and returns a **reverse iterator object** (does not create a new reversed list).
- To obtain a reversed list, wrap the result with `'list()'`.
- Useful for looping over sequences backwards without copying data.

slice(start=0, stop, step=1)

- Creates a slice object representing indices in a sequence.
- Can be applied to any ordered sequence via square bracket indexing.
- Enables reusable slice patterns (e.g., always first 5 elements).
- Parameters:
 - `start`: starting index (default 0)
 - `stop`: stopping index (required)
 - `step`: step size (default 1)

iter(iterable) and next(iterator)

- `iter()` returns an iterator from a sequence.
- `next()` retrieves the next element from the iterator.
- When iterator is exhausted, `next()` raises `StopIteration` exception.
- Enables manual control over iteration process.

Section 3: Example Code and Use Cases

Example of sorted()

```
list1 = [1, 12, 7, -3, 8]
print(sorted(list1)) # Output: [-3, 1, 7, 8, 12]

print(sorted(list1, reverse=True)) # Output: [12, 8, 7, 1, -3]

print(sorted(list1, key=abs)) # Output: [1, -3, 7, 8, 12]
```

Sorts list in ascending order, descending order, and by absolute value ignoring sign.

Example of reversed()

```
list1 = [1, 12, 7, -3, 8]
rev = reversed(list1) # returns a reverse iterator
print(list(rev)) # Output: [8, -3, 7, 12, 1]
```

`reversed()` returns an iterator, so to get a list you convert it with `list()`.

Example of slice()

```
list1 = [10, 20, 30, 40, 50, 60, 70]
s = slice(5) # slice object for first 5 elements
print(list1[s]) # Output: [10, 20, 30, 40, 50]
```

Creating and reusing a slice object to get first 5 elements.

Example of `iter()` and `next()`

`next(it)` if called again raises `StopIteration` error

```
list1 = [10, 20]
it = iter(list1)    # create iterator

print(next(it))     # Output: 10 (first element)
print(next(it))     # Output: 20 (second element)
```

Manually iterating over a list using iterator and `next()` function.

Section 4: Key Takeaways

- **`sorted()`** returns a new sorted list from any iterable; supports reverse sorting and custom key functions.
- **`reversed()`** returns a reverse iterator, not a reversed list; convert to a list if needed.
- The **`slice()`** function creates slice objects for slicing sequences, enabling reusable slice operations.
- **`iter()`** creates an iterator for manual iteration, and **`next()`** fetches elements until exhaustion.
- These built-in functions provide powerful, memory-efficient ways to manipulate and iterate over sequences.
- Practice with these functions helps deepen understanding of Python iteration and sequence handling.